

EXP NO. 1(sup/unsup/reinfo)

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

Step 2: Load Dataset

```
df = pd.read_csv('/content/Mall_Customers (1).csv')
```

Step 3: Display first 5 rows

```
print(df.head())
```

Step 4: Select Features for Clustering

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

Step 5: Feature Scaling

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Step 6: Find Optimal Clusters using Elbow Method

```
wcss = []
```

```
for i in range(1, 11):  
    kmeans = KMeans(n_clusters=i, random_state=42)  
    kmeans.fit(X_scaled)  
    wcss.append(kmeans.inertia_)
```

```
# Plot Elbow Graph
```

```
plt.figure(figsize=(8, 5))  
plt.plot(range(1, 11), wcss, marker='o')
```

```
plt.title('Elbow Method')  
plt.xlabel('Number of Clusters')  
plt.ylabel('WCSS')
```

```
plt.show()
```

```
# Step 7: Apply K-Means
```

```
kmeans = KMeans(n_clusters=5, random_state=42)  
clusters = kmeans.fit_predict(X_scaled)
```

```
# Add cluster column
```

```
df['Cluster'] = clusters
```

```
print(df.head())
```

```
# Step 8: Visualize Clusters
```

```
plt.figure(figsize=(8, 6))

sns.scatterplot(
    x=df['Annual Income (k$)'],
    y=df['Spending Score (1-100)'],
    hue=df['Cluster'],
    palette='Set1',
    s=100
)

plt.title('Customer Segments')
plt.show()
```

EXP NO.2(Spam Email Detection using Naive Bayes)

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# 1. Sample Dataset
data = {
    "email": [
```

```

    "Congratulations! You won a free lottery ticket",
    "Hi John, can we meet tomorrow?",
    "Claim your free prize now",
    "Please review the attached report",
    "Win cash now!!",
    "Let's schedule a meeting",
    "Get free entry into our contest",
    "Project deadline is tomorrow",
    "Earn money fast and easy",
    "Lunch at 1 pm?",
    "You have won $5000! Click here to claim now"
],

"label": [
    "spam", "ham", "spam", "ham", "spam",
    "ham", "spam", "ham", "spam", "ham", "spam"
]
}

```

```
df = pd.DataFrame(data)
```

2. Train-Test Split

```

X_train, X_test, y_train, y_test = train_test_split(
    df["email"],
    df["label"],
    test_size=0.3,

```

```
random_state=42
)

# 3. Convert Text to Numbers
vectorizer = TfidfVectorizer(stop_words="english")

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# 4. Train Model
model = MultinomialNB()
model.fit(X_train_vec, y_train)

# 5. Evaluate Model
y_pred = model.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# 6. Test with New Email
new_email = ["You have won $5000! Click here to claim now"]

new_email_vec = vectorizer.transform(new_email)
```

```
prediction = model.predict(new_email_vec)
```

```
print("\nNew Email Prediction:", prediction[0])
```

EXP NO.3(Data Preprocessing using MPG Dataset)

```
# Import necessary libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
# Load dataset
```

```
import seaborn as sns
```

```
df = sns.load_dataset('mpg')
```

```
# Display dataset
```

```
print(df)
```

```
# Statistical summary
```

```
print(df.describe())
```

```
# Display last 10 rows
```

```
print(df.tail(10))
```

```
# Check missing values
print("Missing values:\n", df.isnull().sum())

# Replace missing values in horsepower with mean
df['horsepower'] = df['horsepower'].fillna(df['horsepower'].mean())

# Encoding categorical variables
encoder = LabelEncoder()

df['origin'] = encoder.fit_transform(df['origin'])
df['name'] = encoder.fit_transform(df['name'])

# Defining features and target variable
X = df.drop(columns=['mpg'])
y = df['mpg']

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Feature scaling
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# Display processed training data
```

```
print(
```

```
    "Processed Training Data Sample:\n",
```

```
    pd.DataFrame(X_train_scaled, columns=X.columns).head()
```

```
)
```

EXP NO . 4(PCA)

```
# Import necessary libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
# Load the MPG Dataset
```

```
mpg_df = sns.load_dataset('mpg')
```

```
# Display statistical summary
```

```
print(mpg_df.describe())
```



```
# Handling missing values

# Replacing NaN values in horsepower with mean value
mpg_df['horsepower'].fillna(
    mpg_df['horsepower'].mean(),
    inplace=True
)

# Encoding categorical variables
encoder = LabelEncoder()

mpg_df['origin'] = encoder.fit_transform(mpg_df['origin'])
mpg_df['name'] = encoder.fit_transform(mpg_df['name'])

# Selecting features for PCA
X = mpg_df.drop(columns=['mpg'])

# Standardizing the data
# PCA is sensitive to scale
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Applying PCA
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
# Convert PCA results into DataFrame
```

```
pca_df = pd.DataFrame(  
    X_pca,  
    columns=['PC1', 'PC2']  
)
```

```
# Explained variance ratio
```

```
print("Explained Variance Ratio:")  
print(pca.explained_variance_ratio_)
```

```
# Scatter plot of PCA components
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(  
    pca_df['PC1'],  
    pca_df['PC2'],  
    alpha=0.7,  
    c=mpg_df['origin'],  
    cmap='viridis'  
)
```

```
plt.colorbar(label='Origin')
```

```
plt.xlabel('Principal Component 1')
```

```
plt.ylabel('Principal Component 2')
```

```
plt.title("PCA of MPG Dataset")
```

```
plt.show()
```

EXP NO.5(Iris Flower Species Classification)

```
# Importing Dependencies
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Load Dataset
```

```
df = pd.read_csv("/content/E05_Iris-flower_Dataset.csv")
```

```
# Display dataset
```

```
print(df)
```

```
# Shape of dataset
```

```
print(df.shape)
```

```
# Statistical Summary
```

```
print(df.describe())
```

```
# Mean values grouped by Species
print(df.groupby('Species').mean())
```

```
# Scatter Plot
```

```
sns.scatterplot(
    x='SepalLengthCm',
    y='SepalWidthCm',
    hue='Species',
    data=df
)
```

```
plt.show()
```

```
# Line Plot
```

```
sns.lineplot(data=df.drop(['Species'], axis=1))
plt.show()
```

```
# Remove Species column
```

```
df1 = df.drop('Species', axis=1)
```

```
print(df1)
```

```
# KDE Plot
```

```
g = sns.FacetGrid(df, col='Species')
g = g.map(sns.kdeplot, 'SepalLengthCm')
```

```
# Pair Plot
```

```
sns.pairplot(df)
```

```
# Drop Id column
```

```
df.drop('Id', axis=1, inplace=True)
```

```
# MACHINE LEARNING
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.linear_model import LogisticRegression
```

```
# Features and Target
```

```
X = df.drop('Species', axis=1)
```

```
y = df['Species']
```

```
# Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.3,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Logistic Regression
```

```
logreg = LogisticRegression()
```

```
logreg.fit(X, y)
```

```
y_pred = logreg.predict(X_test)
```

```
print(accuracy_score(y_test, y_pred))
```

```
# Save Model
```

```
import pickle
```

```
with open("iris_model.pkl", "wb") as file:
```

```
    pickle.dump(logreg, file)
```

```
print("Model saved successfully!")
```

```
# Load Model
```

```
model = pickle.load(open("iris_model.pkl", "rb"))
```

```
# User Input
```

```
sepal_length = float(input("Enter Sepal Length: "))
```

```
sepal_width = float(input("Enter Sepal Width: "))
```

```
petal_length = float(input("Enter Petal Length: "))
```

```
petal_width = float(input("Enter Petal Width: "))
```

```
# Create DataFrame
```

```
features_df = pd.DataFrame(  
    [[sepal_length, sepal_width, petal_length, petal_width]],  
    columns=[  
        'SepalLengthCm',  
        'SepalWidthCm',  
        'PetalLengthCm',  
        'PetalWidthCm'  
    ]  
)
```

```
# Predict Species
```

```
prediction = model.predict(features_df)
```

```
print("Predicted Species:", prediction[0])
```

EXP NO 6(House Price Prediction)

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.preprocessing import OrdinalEncoder
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction import DictVectorizer
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Load Dataset
df = pd.read_csv("/content/E06_house price data less.csv")

# Display first 10 rows
print(df.head(10))

# Dataset Shape
print(df.shape)

# Check missing values
print(df.isnull().sum())

# Data types
print(df.dtypes)

# Statistical Summary
print(df.describe())

# Pre-processing
# Drop unnecessary columns
```



```
df.drop('ID', axis=1, inplace=True)
df.drop('Locality', axis=1, inplace=True)
```

```
# Ordinal Encoding
```

```
ordinal_columns = [
    'Property_Type',
    'Furnished_Status',
    'Public_Transport_Accessibility',
    'Facing',
    'Security'
]
```

```
categories = [
    ['Apartment', 'Independent House', 'Villa'],
    ['Unfurnished', 'Semi-furnished', 'Furnished'],
    ['Low', 'Medium', 'High'],
    ['South', 'East', 'West', 'North'],
    ['No', 'Yes']
]
```

```
encoder = OrdinalEncoder(categories=categories)
```

```
df[ordinal_columns] = encoder.fit_transform(df[ordinal_columns])
```

```
# Separate categorical and numerical columns
```

```
categorical_col = [
```

```
    column for column in df.columns
    if df[column].dtype == 'object'
]
```

```
numerical_col = [
    column for column in df.columns
    if df[column].dtype != 'object'
]
```

```
# Convert categorical text to lowercase
for column in categorical_col:
    df[column] = df[column].str.lower()
```

```
# Correlation Matrix
correlation_matrix = df[numerical_col].corr()
```

```
# Heatmap
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(
    correlation_matrix,
    cmap='coolwarm',
    fmt=".2f",
    linewidths=0.5
)
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

```
# Randomly sample 40% of the data
```

```
df_sampled = df.sample(frac=0.4, random_state=1)
```

```
# Features and Target
```

```
X = df_sampled.drop('Price_in_Lakhs', axis=1)
```

```
y = df_sampled['Price_in_Lakhs']
```

```
# Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=1
```

```
)
```

```
# Convert to dictionary format
```

```
train_dicts = X_train.to_dict(orient='records')
```

```
test_dicts = X_test.to_dict(orient='records')
```

```
# Encode categorical variables
```

```
dv = DictVectorizer(sparse=False)
```

```
X_train = dv.fit_transform(train_dicts)
```

```
X_test = dv.transform(test_dicts)

# Train Linear Regression Model
model = LinearRegression()

model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation Function
def evaluate_model(y, y_pred):

    mae = mean_absolute_error(y, y_pred)
    mse = mean_squared_error(y, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y, y_pred)

    return mae, mse, rmse, r2

# Evaluate Model
mae, mse, rmse, r2 = evaluate_model(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
```

```
print("R2 Score:", r2)
```

EXP NO. 7(Markov process)

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import networkx as nx
```

```
class ManufacturingProcess:
```

```
    def __init__(self, trans_mat, init_stt, stt):
```

```
        self.trans_mat = trans_mat
```

```
        self.curr_stt = init_stt
```

```
        self.stt = stt
```

```
        self.hist = [init_stt]
```

```
    def step(self):
```

```
        self.curr_stt = np.random.choice(
```

```
            len(self.stt),
```

```
            p=self.trans_mat[self.curr_stt]
```

```
        )
```

```
        self.hist.append(self.curr_stt)
```

```
    def sim(self, num_stp):
```

```
        print("Simulation Started")
```

```
print("Initial State:", self.stt[self.curr_stt])
```

```
for i in range(num_stp):
```

```
    self.step()
```

```
    print(f"Step {i+1}: {self.stt[self.curr_stt]}")
```

```
    # Stop if Finished Product reached
```

```
    if self.curr_stt == 2:
```

```
        print("Reached Finished Product.")
```

```
        break
```

```
print("Simulation Finished.")
```

```
def plot_sim(self):
```

```
    plt.plot(self.hist, marker='o')
```

```
    plt.yticks([0, 1, 2], self.stt)
```

```
    plt.xlabel("Steps")
```

```
    plt.ylabel("State")
```

```
    plt.title("State Transition Over Time")
```

```
plt.grid()
```

```
plt.show()
```

```
def draw_graph(self):
```

```
    G = nx.DiGraph()
```

```
    # Add edges
```

```
    for i in range(len(self.stt)):
```

```
        for j in range(len(self.stt)):
```

```
            prob = self.trans_mat[i][j]
```

```
            if prob > 0:
```

```
                G.add_edge(
```

```
                    self.stt[i],
```

```
                    self.stt[j],
```

```
                    weight=prob
```

```
                )
```

```
    pos = nx.spring_layout(G, seed=42)
```

```
    nx.draw(
```

```
        G,
```

```
        pos,
```

```

        with_labels=True,
        node_color='lightblue',
        node_size=3000,
        font_size=10
    )

    edge_labels = {
        (self.stt[i], self.stt[j]): f"{self.trans_mat[i][j]:.2f}"
        for i in range(len(self.stt))
        for j in range(len(self.stt))
        if self.trans_mat[i][j] > 0
    }

    nx.draw_networkx_edge_labels(
        G,
        pos,
        edge_labels=edge_labels
    )

    plt.title("Manufacturing Process - Markov Chain")

    plt.show()

# Transition Matrix
trans_mat = np.array([
    [0.7, 0.2, 0.1],

```



```
[0.1, 0.6, 0.3],  
[0.0, 0.0, 1.0]  
]  
  
# States  
stt = [  
    "Raw Material",  
    "Work in Progress",  
    "Finished Product"  
]  
  
# Initial State  
init_stt = 0  
  
# Create Object  
process = ManufacturingProcess(  
    trans_mat,  
    init_stt,  
    stt  
)  
  
# Run Simulation  
process.sim(20)  
  
# Plot State Transitions  
process.plot_sim()
```

```
# Draw Markov Chain Graph
```

```
process.draw_graph()
```

